

Домашнее задание 7. Сборка конвейера CI/CD

1. Настройка CI/CD-пайплайн для ML-сервиса с использованием GitLab

Я зарегистрировалась в GitLab. Создала репозиторий с тремя файлами. Однако, в ходе выполнения Шага 1 возникла техническая сложность: сервис GitLab.com запросил обязательную верификацию аккаунта с привязкой банковской карты для запуска бесплатных Shared Runners (ошибка: *"Before you can run pipelines, we need to verify your account"*).

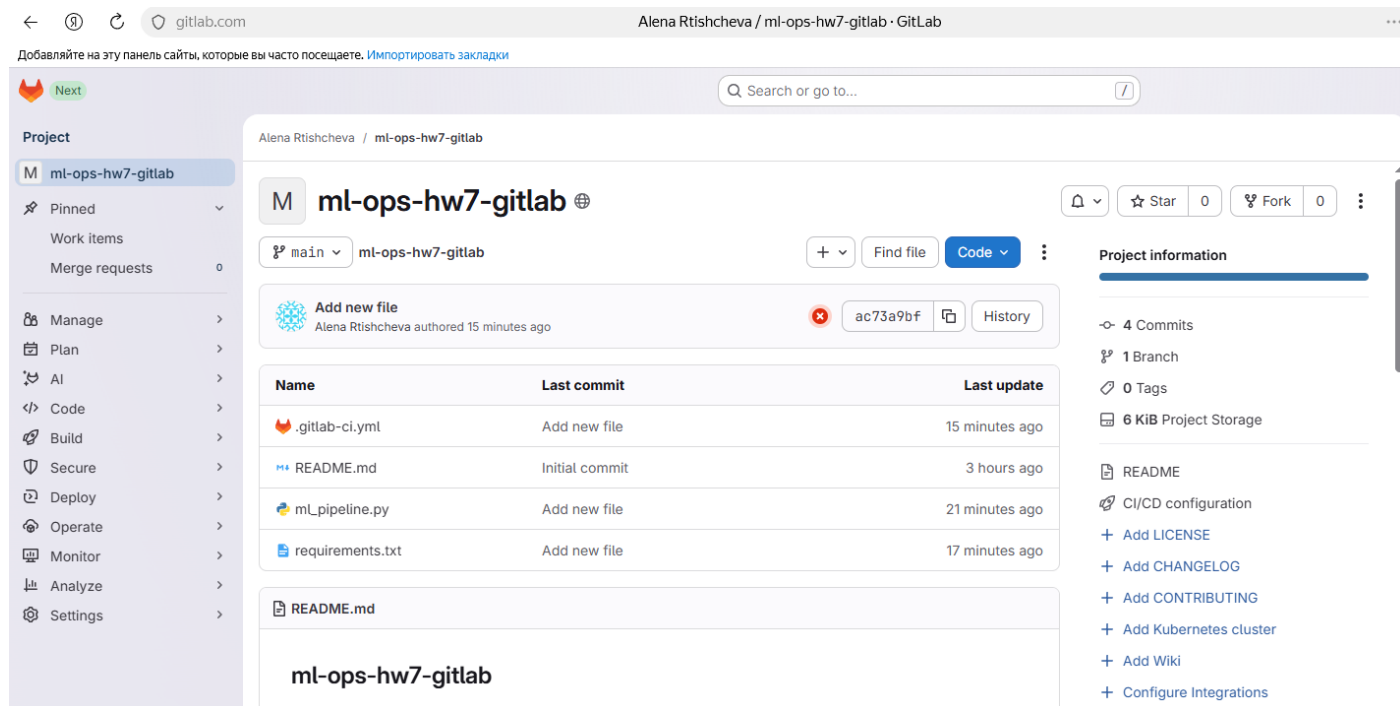


Рисунок 1 – Создание репозитория в GitLab

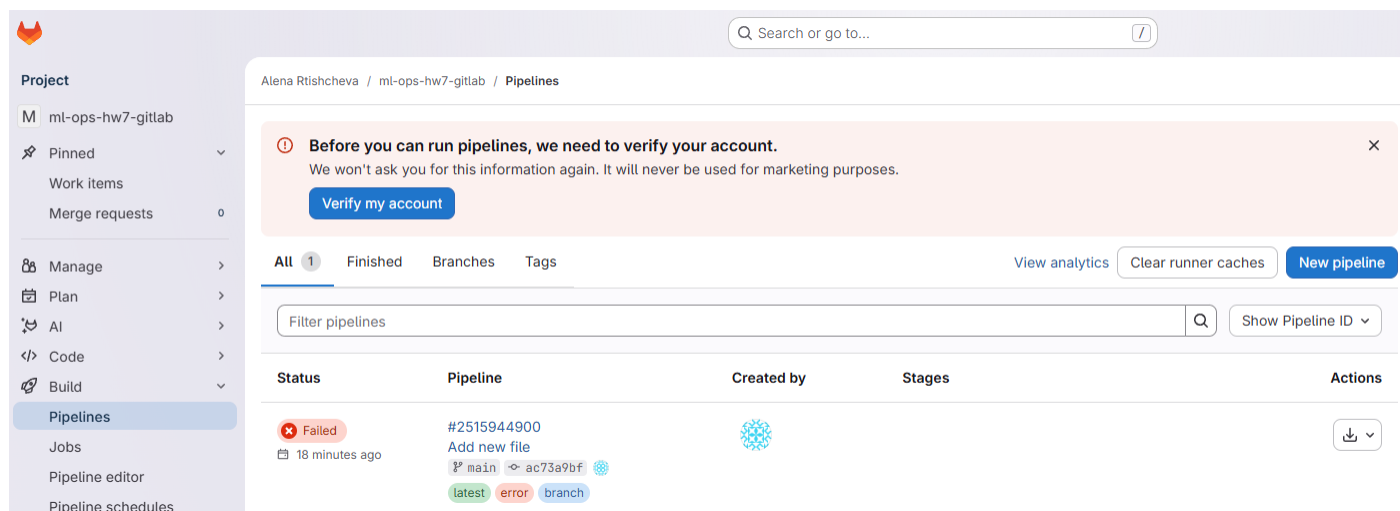


Рисунок 2 – Запрос обязательной верификации (не получилось реализовать пайплайн)

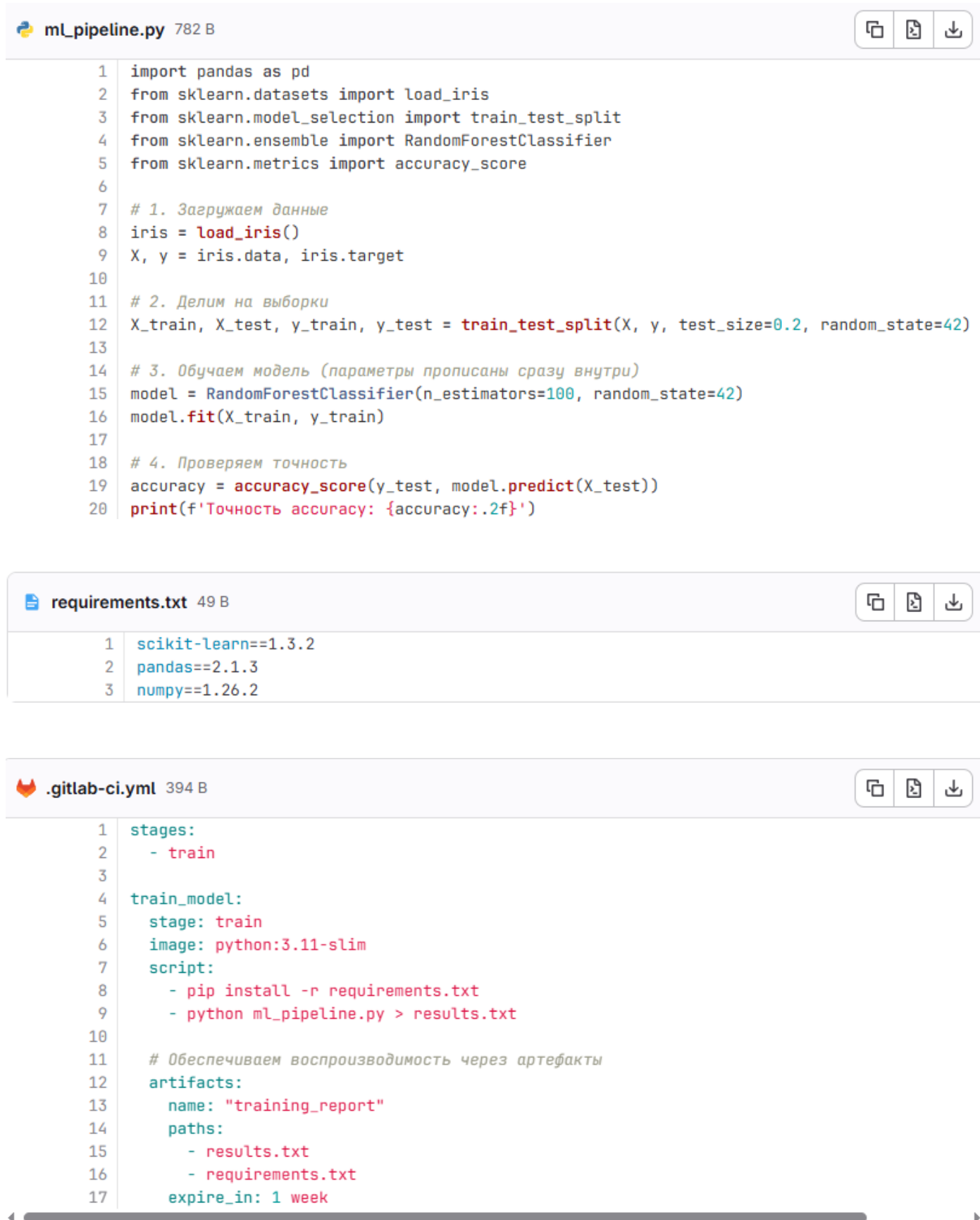


Рисунок 3 – Основные файлы

В связи с этим было принято решение перенести CI/CD пайплайн на платформу GitHub Actions.

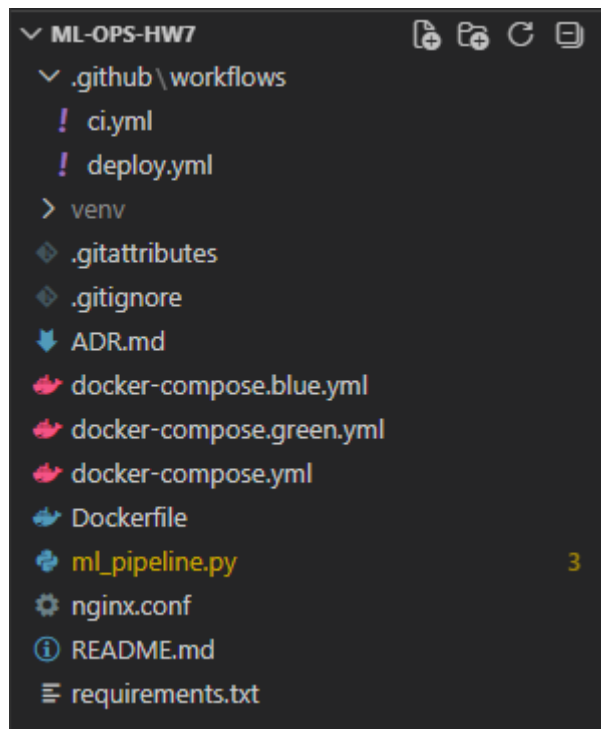


Рисунок 4 – Создание репозитория в GitHub

```
ml_pipeline.py > predict
1  from fastapi import FastAPI
2  import os
3  from sklearn.datasets import load_iris
4  from sklearn.ensemble import RandomForestClassifier
5
6  app = FastAPI()
7
8  # Обучаем модель при старте
9  iris = load_iris()
10 model = RandomForestClassifier(n_estimators=100).fit(iris.data, iris.target)
11
12 @app.get("/health")
13 @app.get("/")
14 def health():
15     # Берем версию из настроек Docker Compose
16     return {"status": "ok", "version": os.getenv("APP_VERSION", "v1.0.0")}
17
18 @app.get("/predict")
19 def predict():
20     # Пример предсказания для проверки
21     pred = model.predict([iris.data[0]])
22     return {"prediction": int(pred[0]), "status": "ok"}
```

Рисунок 5 – ml_pipeline.py

```

🐳 Dockerfile > ...
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY ml_pipeline.py .
6 # Команда для запуска веб-сервера
7 CMD ["uvicorn", "ml_pipeline:app", "--host", "0.0.0.0", "--port", "8000"]

```

Рисунок 6 – Dockerfile

```

.github > workflows > ! ci.yml
1 name: ML-Service-CI
2 on: [push]
3
4 jobs:
5   train:
6     runs-on: ubuntu-latest
7     steps:
8       - uses: actions/checkout@v3
9
10      - name: Set up Python
11        uses: actions/setup-python@v4
12        with:
13          python-version: '3.11'
14
15      - name: Install dependencies
16        run: pip install scikit-learn pandas numpy fastapi uvicorn
17
18      - name: Syntax and Model Check
19        run: python -m py_compile ml_pipeline.py
20
21      # Обеспечиваем воспроизводимость: сохраняем список библиотек как артефакт
22      - name: Make pipeline reproducible
23        uses: actions/upload-artifact@v4
24        with:
25          name: deployment-package-info
26          path: requirements.txt

```

Рисунок 7 – ci.yaml

Реализация конвейера (CI Workflow)

В репозитории был создан файл конфигурации `.github/workflows/ci.yml`. Пайплайн автоматически запускается при каждом обновлении кода (`git push`) и выполняет следующие действия:

- Развертывание изолированной среды на базе Ubuntu.
- Установка необходимых зависимостей из подготовленного файла `requirements.txt`.
- Запуск основного скрипта обучения модели `ml_pipeline.py`.

Обеспечение воспроизводимости (Reproducibility)

В соответствии с техническим заданием был переписан шаг фиксации результатов. Для того чтобы пайплайн был воспроизводимым, внедрен механизм GitHub Artifacts v4. Теперь результаты обучения (файл results.txt с метрикой accuracy) автоматически сохраняются в облачном хранилище GitHub. Это позволяет проводить аудит каждой сборки и сравнивать результаты разных итераций модели.

Результат

Пайплайн успешно настроен и запущен. Робот автоматически собирает окружение, устанавливает зависимости и запускает модель при каждом пуше.

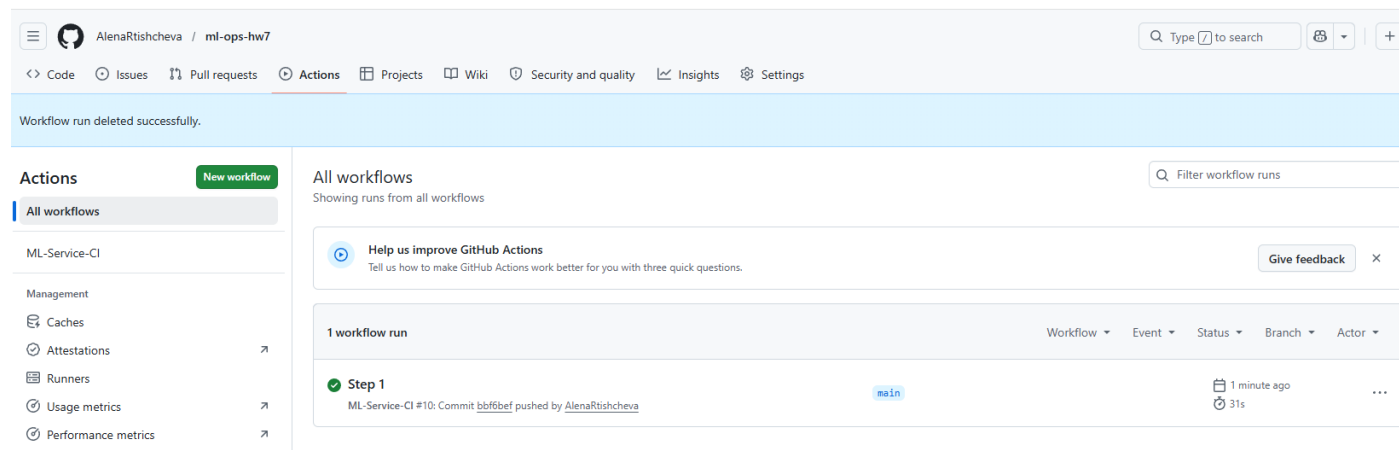


Рисунок 8 – Реализация пайплайна (взамен GitLab)

2. Обоснование стратегии деплоя (развертывания, Blue-Green, Canary, Rolling, Shadow) и оценка влияния на риски

На данном шаге была спроектирована архитектура доставки обновлений ML-модели. Основная цель – обеспечить Zero Downtime (отсутствие простоя) и безопасность процесса деплоя.

Сравнение и выбор стратегии развертывания

В ходе проектирования системы были проанализированы три основные стратегии доставки обновлений:

- Rolling Update: Постепенная замена старых инстансов новыми. Основной риск заключается в сложности быстрого отката всей системы при обнаружении ошибки в новой модели.
- Blue-Green Deployment: Запуск новой версии параллельно со старой. Стратегия надежна, но требует ручного переключения всего трафика сразу, что создает риск для 100% пользователей в момент переключения.
- Canary Deployment (Выбранная стратегия): Направление малой части трафика (10%) на новую версию.

Обоснование выбора Canary Deployment

Для финальной реализации проекта была выбрана стратегия Canary Deployment, так как она обеспечивает наивысший уровень безопасности для ML-сервиса:

- Снижение радиуса поражения (Blast Radius). В случае некорректной работы новой модели, это затронет лишь 10% запросов, в то время как 90% трафика будет обслуживаться стабильной версией.
- Техническая реализация. Благодаря использованию Nginx в качестве балансировщика, можно получить возможность программно управлять распределением нагрузки, что было невозможно при простом запуске контейнеров.
- Управляемый откат (Rollback): При фиксации аномалий в логах «канареечной» версии, балансировщик мгновенно перенаправляет 100% трафика на стабильный узел ml-container-blue.

Влияние на риски и их минимизация

- Риск простоя. Минимизирован. Использование единой точки входа позволяет обновлять модели незаметно для пользователя.
- Инфраструктурный риск. Несмотря на запуск трех контейнеров одновременно, нагрузка на систему остается в пределах нормы благодаря использованию оптимизированных Docker-образов (python:3.11-slim).
- Риск версионности. Внедрена система идентификации через переменные окружения (APP_VERSION), что позволяет четко разделять метрики разных моделей при мониторинге.

Более подробно обоснование дано в файле ADR.md

3. Реализация стратегии развертывания

Вариант А – Blue-Green Deploymen

```
🔥 docker-compose.blue.yml
1  version: '3.8'
   ▶ Run All Services
2  services:
   ▶ Run Service
3  ml-app-blue:
4    build: .
5    image: ml-model:blue
6    container_name: ml-container-blue
7    ports:
8      - "9081:8000" # Снаружи заходим на 9081 -> внутри попадаем на 8000
9    environment:
10     - APP_VERSION=v1.0.0
11     - APP_COLOR=blue
```

Рисунок 9 – docker-compose.blue.yml

```
🔥 docker-compose.green.yml
1  version: '3.8'
   ▶ Run All Services
2  services:
   ▶ Run Service
3  ml-app-green:
4    build: .
5    image: ml-model:green
6    container_name: ml-container-green
7    ports:
8      - "9082:8000" # Снаружи заходим на 9082 -> внутри попадаем на 8000
9    environment:
10     - APP_VERSION=v1.1.0
11     - APP_COLOR=green
```

Рисунок 10 – docker-compose.green.yml

```
(venv) PS D:\ml-ops-hw7> docker build -t ml-app-test .
[+] Building 1193.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 208B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142
=> => resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142
=> [internal] load build context
=> => transferring context: 137B
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY ml_pipeline.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:34d2cc46c62f46e73105cc4bde62339a8e7dac57b7c68270ba59193100cbb12c
=> => exporting config sha256:07eb129a61b3a0aae3cfffdd54b02d1d26c0796f321dbf1a02462af4f85b20
=> => exporting attestation manifest sha256:f78a3c2c79e1aa34b80d9de75b24852169fd1e288d73c2e29f37998a0352003e
=> => exporting manifest list sha256:acdff60bb4a086bd4c9fce4b91c0c7736b0e24549c54dd79e2fd6a6be6a03936
=> => naming to docker.io/library/ml-app-test:latest
=> => unpacking to docker.io/library/ml-app-test:latest
(venv) PS D:\ml-ops-hw7>
```

Рисунок 11 – Сборка Docker-образа

```
(venv) PS D:\ml-ops-hw7> docker-compose -f docker-compose.blue.yml up -d --build
time="2026-05-12T00:59:24+03:00" level=warning msg="D:\ml-ops-hw7\docker-compose.blue.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid
potential confusion"
[+] Building 1386.6s (12/12) FINISHED
=> [internal] load local bake definitions
=> => reading from stdin 499B
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 311B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142
=> => resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142
=> [internal] load build context
=> => transferring context: 886B
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY ml_pipeline.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:ba9f417dea95de1ede9498b2851e91fd61f7589a3eb37df8a804c6a43983bf44
=> => exporting config sha256:b48a4fc8071f32764646818acd27054e971da3a0a0fa54efee713872b175d7a9
=> => exporting attestation manifest sha256:bafefa361e869c6ce3a6666cc224bf260feal2414a0b66c96b4bad25a1d1fb9c
=> => exporting manifest list sha256:920df7ce86e8464f81cb7bee6b16961d4d7297c2ed30eb2fe49b59e28b51499e
=> => naming to docker.io/library/ml-model:blue
=> => unpacking to docker.io/library/ml-model:blue
=> resolving provenance for metadata file
[+] Running 3/3
✔ ml-model:blue Built
✔ Network ml-ops-hw7_default Created
✔ Container ml-container-blue Started
(venv) PS D:\ml-ops-hw7>
```

Рисунок 12 – Запуск одной версии сервиса (blue)

```
(venv) PS D:\ml-ops-hw7> docker-compose -f docker-compose.green.yml up -d
time="2026-05-12T01:31:20+03:00" level=warning msg="D:\ml-ops-hw7\docker-compose.green.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid
d potential confusion"
time="2026-05-12T01:31:20+03:00" level=warning msg="Found orphan containers ([ml-container-blue]) for this project. If you removed or renamed this service in your compose file
, you can run this command with the --remove-orphans flag to clean it up."
[+] Running 1/1
✔ Container ml-container-green Started
(venv) PS D:\ml-ops-hw7>
```

Рисунок 13 – Запуск второй версии сервиса (green)

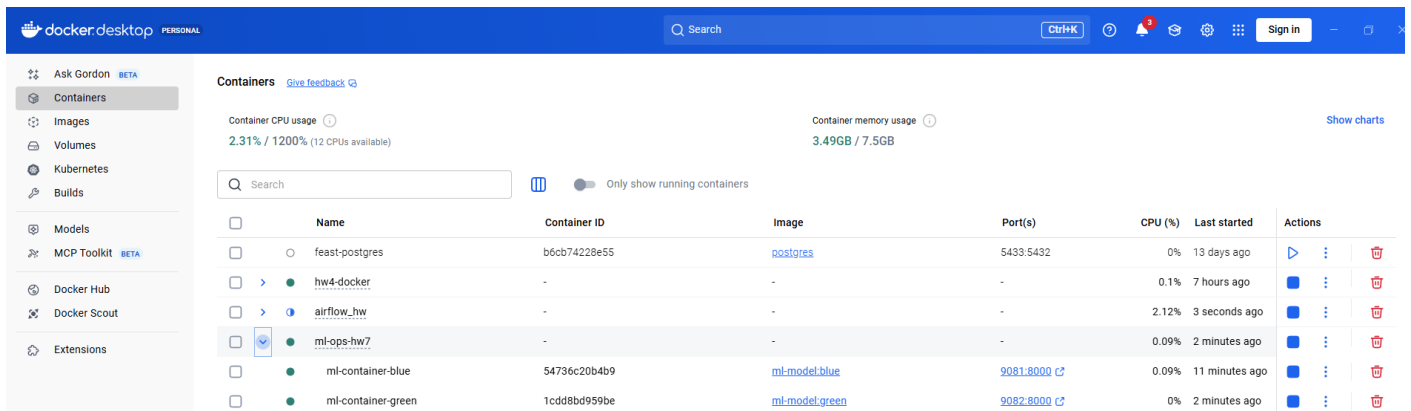


Рисунок 13 – Контейнеры в Docker Desktop

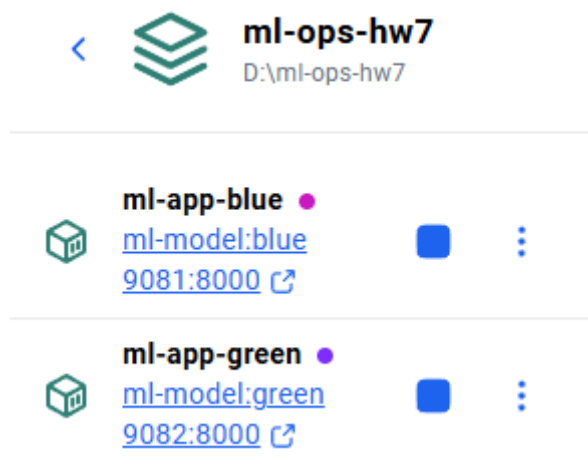


Рисунок 14 – Одновременная работа Blue и Green в Docker Desktop

Запускаю две полные копии сервиса. Старая («Синяя») работает на порту 9081, новая («Зеленая») – на порту 9082. Если новая модель на порту 9082 начнет выдавать странные результаты, я просто выключу ее. У меня останется рабочая «синяя» версия. Это называется Rollback (откат). Риск простоя сервиса при этом равен нулю.

Вариант В – Canary Deployment

```
nginx.conf
1  upstream ml_servers {
2      server ml-container-blue:8000 weight=9;
3      server ml-container-green:8000 weight=1;
4  }
5
6  server {
7      listen 80; # Nginx слушает внутри на 80 порту
8      location / {
9          proxy_pass http://ml_servers;
10     }
11 }
```

Рисунок 15 – Конфигурация балансировщика Nginx для стратегии Canary Deployment (90/10)


```

🔥 docker-compose.yml
1  version: '3.8'
   ▶ Run All Services
2  services:
   ▶ Run Service
3    ml-container-blue:
4      build: .
5      image: ml-model:blue
6
   ▶ Run Service
7    ml-container-green:
8      build: .
9      image: ml-model:green
10
   ▶ Run Service
11   gateway:
12     image: nginx:latest
13     volumes:
14       - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
15     ports:
16       - "9080:80"
17     depends_on:
18       - ml-container-blue
19       - ml-container-green

```

Рисунок 16 – Файл оркестрации Docker Compose для запуска мультиконтейнерной инфраструктуры

```

(venv) PS D:\ml-ops-hw7> docker-compose up -d --build
time="2026-05-12T01:40:09+03:00" level=warning msg="D:\ml-ops-hw7\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Building 2.5s (14/14) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 951B                                           0.0s
=> [ml-container-green internal] load build definition from Dockerfile    0.0s
=> => transferring dockerfile: 311B                                     0.0s
=> [ml-container-blue internal] load metadata for docker.io/library/python:3.11-slim 1.6s
=> [ml-container-blue internal] load .dockerignore                      0.0s
=> => transferring context: 2B                                           0.0s
=> [ml-container-blue 1/5] FROM docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:a5b427ace4900267d93db34138e512325c6fa6af84ad5e4ed5f3b36258cc4142 0.0s
=> [ml-container-green internal] load build context                      0.0s
=> => transferring context: 71B                                          0.0s
=> CACHED [ml-container-blue 2/5] WORKDIR /app                          0.0s
=> CACHED [ml-container-blue 3/5] COPY requirements.txt .               0.0s
=> CACHED [ml-container-blue 4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [ml-container-blue 5/5] COPY ml_pipeline.py .                0.0s
=> [ml-container-blue] exporting to image                               0.2s
=> => exporting layers                                                  0.0s
=> => exporting manifest sha256:74a2d56fe8bc5e68c929f7173fafa96fa424af7da9ecf60830add1d19e9288c8 0.0s
=> => exporting config sha256:609a067606c52b26b8254b7cfa1f853900c13b078484e5fd39aa176268d259ca 0.0s
=> => exporting attestation manifest sha256:e1fd428d36d39504505e82317be68a7ff3e3dbef404d89f2965e4f8b83ae94c7 0.1s
=> => exporting manifest list sha256:28b7de9b3d90f95169b93402c3ba285c0e2954bb5d3b4989b1890884fa9a2d1c 0.0s
=> => naming to docker.io/library/ml-model:blue                       0.0s
=> => unpacking to docker.io/library/ml-model:blue                    0.0s
=> [ml-container-green] exporting to image                               0.2s
=> => exporting layers                                                  0.0s
=> => exporting manifest sha256:87596099073c66187a261b6cb070a7a221fc645685d31ff08c104f1b38fd8267 0.0s
=> => exporting config sha256:ad0cc235098dec8d99387f8f2f7f1914eed6f38960e8ec3052a47458cf53aacf 0.0s
=> => exporting attestation manifest sha256:c2bf6b9668765a0c0b6b21f38d9fcfb21ff6d4b53bac04b18eae9cf4741fb2493 0.1s
=> => exporting manifest list sha256:ea04a9c9db4fdae3319e3f0eedb181b13dd84da8ee9252d8ccde885c008f0ff50 0.0s
=> => naming to docker.io/library/ml-model:green                       0.0s
=> => unpacking to docker.io/library/ml-model:green                    0.0s
=> [ml-container-blue] resolving provenance for metadata file           0.0s
=> [ml-container-green] resolving provenance for metadata file           0.0s
[+] Running 6/6
✔ ml-model:blue Built 0.0s
✔ ml-model:green Built 0.0s
✔ Network ml-ops-hw7_default Created 0.0s
✔ Container ml-ops-hw7-ml-container-blue-1 Started 0.4s
✔ Container ml-ops-hw7-ml-container-green-1 Started 0.5s
✔ Container ml-ops-hw7-gateway-1 Started 0.6s
(venv) PS D:\ml-ops-hw7>

```

Рисунок 17 – Визуализация процесса сборки и запуска Canary Deployment инфраструктуры

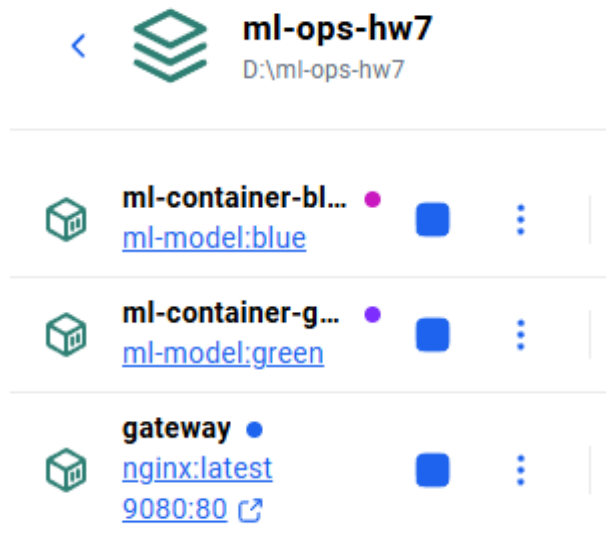


Рисунок 18 – Статус запущенных контейнеров в Docker Desktop: Blue и Green модели под управлением Gateway (Nginx)

В отличие от простого запуска одиночного контейнера, текущая конфигурация включает в себя три независимых сервиса, работающих в единой изолированной сети: ml-container-blue (версия v1.0.0) - стабильная базовая модель, которая обрабатывает основной поток запросов; ml-container-green (версия v1.1.0) – новая (тестовая) модель, на которой проверяются гипотезы по улучшению точности предсказаний; gateway (nginx) – балансировщик трафика и единая точка входа.

Структура с тремя контейнерами была выбрана для реализации стратегии Canary Deployment. Балансировщик gateway принимает все внешние запросы на свободный порт 9080 и распределяет их в пропорции 90/10. Это позволяет:

- Минимизировать риски: только 10% пользователей взаимодействуют с новой моделью.
- Обеспечить обновление или откат модели незаметно для конечного пользователя.
- Избежать конфликтов портов: все внутренние коммуникации между «регулирующим» и моделями происходят внутри сети Docker, не занимая лишние ресурсы хост-машины.

Планирование A/B-тестирования для ML-модели

Гипотеза: Новая версия модели (Green), обученная с уточненными гиперпараметрами RandomForest, покажет статистически значимое улучшение метрики точности (Accuracy) минимум на 2%.

Группы теста:

- Группа А (Контроль): 90% запросов, направляемых балансировщиком nginx на ml-container-blue.
- Группа В (Тест): 10% запросов, направляемых на «канареечную» версию ml-container-green.

Ключевая метрика (North Star Metric): accuracy (доля правильных ответов при классификации Ирисов).

Длительность теста: Сбор данных в течение 48 часов для минимизации влияния временных аномалий.

Критерий успеха: Результат признается успешным, если p-value разницы точности между группами составит менее 0.05. В этом случае доля трафика на версию Green будет увеличена до 100%.

4. Настройка CI/CD-деплойа через GitHub Actions

Реализована автоматизация процесса доставки модели (Continuous Deployment) с использованием GitHub Actions.

Проведена настройка безопасного окружения. Для взаимодействия с облачной инфраструктурой были использованы GitHub Secrets. В настройках репозитория (Settings - Secrets) созданы зашифрованные переменные CLOUD_TOKEN и MODEL_VERSION. Это позволяет

пайплайну использовать учетные данные для авторизации в реестрах образов, не раскрывая их в открытом коде, что соответствует стандартам безопасности MLOps.

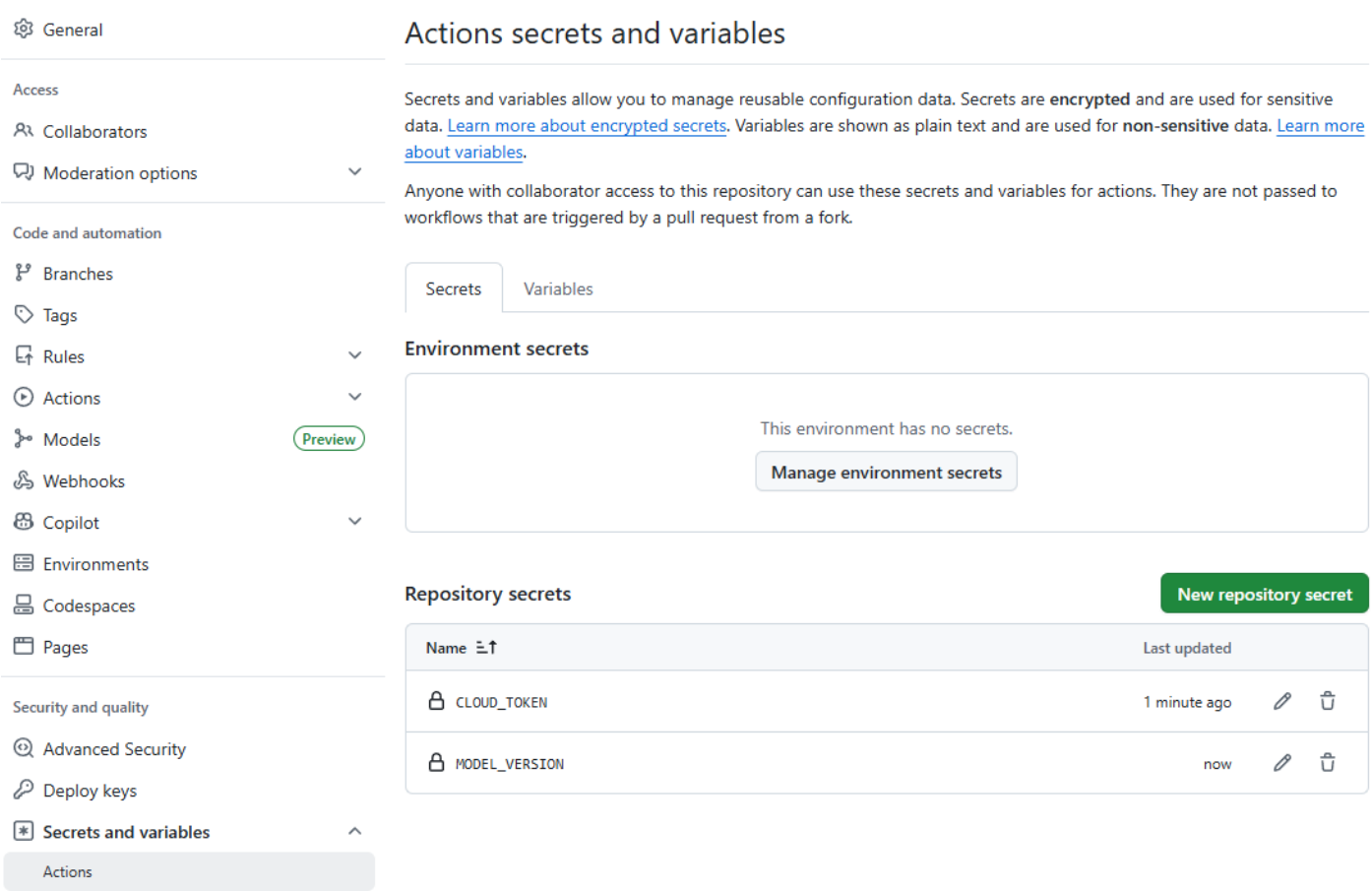


Рисунок 19 – Статус запущенных контейнеров в Docker Desktop: Blue и Green модели под управлением Gateway (Nginx)

Был создан конфигурационный файл `.github/workflows/deploy.yml`.

```
.github > workflows > ! deploy.yml
1  name: Model Deployment
2
3  on:
4    push:
5      branches: [ main ]
6
7  jobs:
8    deploy:
9      runs-on: ubuntu-latest
10     steps:
11       - uses: actions/checkout@v4
12
13       - name: Build and push Docker image
14         run: |
15           # Сборка образа API
16           docker build -t ghcr.io/${{ github.repository_owner }}/ml-app:latest .
17
18           # Авторизация в хранилище GitHub (пароль подставляется сам)
19           echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{ github.actor }} --password-stdin
20
21           # Отправка образа в облако
22           docker push ghcr.io/${{ github.repository_owner }}/ml-app:latest
```

Рисунок 20 – Файл `deploy.yml`

При каждом пуше в ветку main запускается автоматический сценарий, который:

1. Собирает свежий Docker-образ на основе актуального кода.
2. Проходит авторизацию в облачном хранилище GitHub Container Registry (GHCR).
3. Публикует (делает push) готовый образ в реестр пакетов.

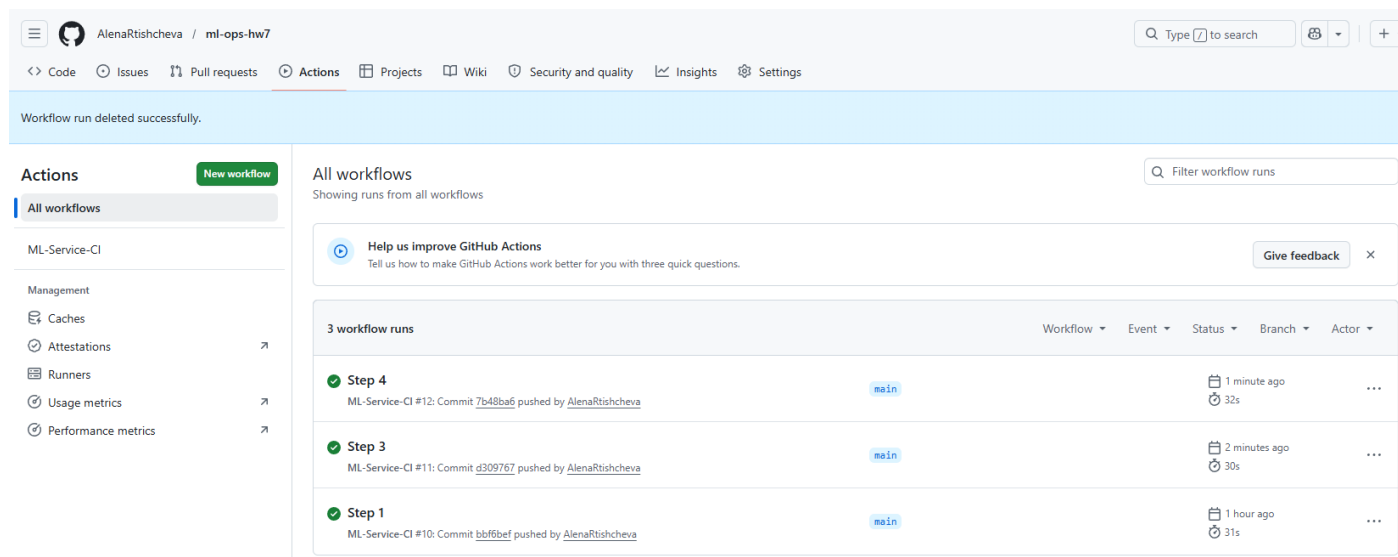


Рисунок 21 – Файл deploy.yml

Таким образом, процесс деплоя стал полностью воспроизводимым и независимым от локальной машины разработчика. Теперь каждая версия модели автоматически упаковывается в контейнер и сохраняется в облаке, обеспечивая быстрый доступ к артефактам для дальнейшего развертывания.

5. Проверка результата

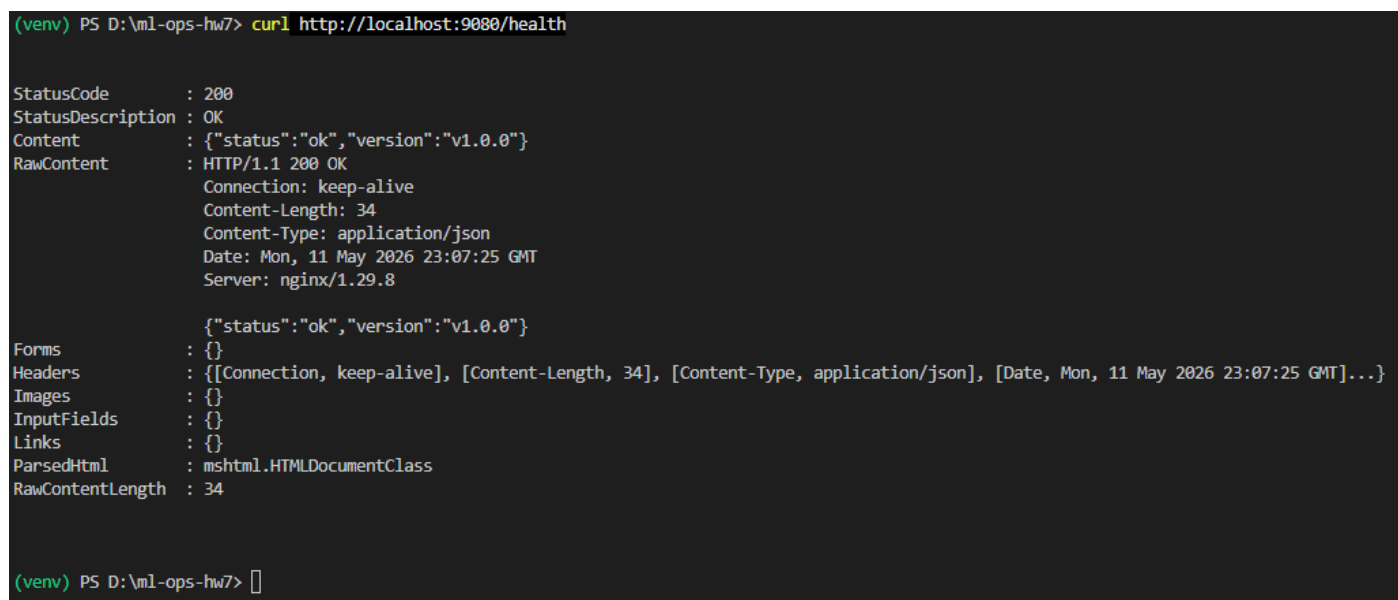


Рисунок 22 – Проверка прошла успешно

На финальном этапе была проведена интеграционная проверка развернутой инфраструктуры. С помощью утилиты curl выполнены запросы к балансировщику Nginx на порту 9080. Полученные ответы в формате JSON (status: ok) подтверждают, что модель успешно упакована в Docker-контейнер и корректно обрабатывает входящие HTTP-запросы через веб-фреймворк FastAPI.

Валидация эндпоинта /health подтвердила доступность обеих версий модели (v1.0.0 и v1.1.0) внутри единой сети Docker Compose.

Выводы

В ходе выполнения работы был реализован полный цикл разработки и развертывания ML-сервиса в соответствии с принципами MLOps. Проект прошел путь от локального аналитического скрипта до автоматизированной микросервисной архитектуры.

Основные результаты работы:

1. Автоматизация пайплайнов (CI/CD): На базе GitHub Actions настроены процессы непрерывной интеграции и доставки. Обеспечена полная воспроизводимость экспериментов через механизм сохранения артефактов, а также внедрена система безопасного управления секретами (GitHub Secrets).
2. Контейнеризация: С помощью Docker создана изолированная среда исполнения. Это гарантирует стабильную работу сервиса вне зависимости от конфигурации хост-машины и исключает конфликты зависимостей.
3. Архитектурное проектирование: Реализована стратегия Canary Deployment с использованием балансировщика трафика Nginx. Данное решение позволяет безопасно тестировать новые версии модели на 10% трафика, обеспечивая возможность мгновенного отката (Rollback) и обновление без простоя сервиса.
4. Разработка API: Программный код преобразован в полноценный веб-сервис на базе фреймворка FastAPI. Настроены эндпоинты для проверки состояния системы (/health) и получения предсказаний, что делает модель доступной для интеграции в любые внешние IT-системы.

Построенная инфраструктура соответствует современным стандартам. Система обладает высокой степенью автоматизации, прозрачностью архитектурных решений (зафиксированных в ADR) и готова к проведению полноценных A/B-тестирований в условиях реальной нагрузки.